# Reconciliation Exception Classification Review

Prepared for the reconciliation-platform workspace. Review date: April 5, 2026. Scope: full reconciliation flow from upload and workbook parsing through normalization, matching, exception classification, summary generation, workbook export, API delivery, and web presentation. This rerun reflects the current grouped-residual enhancements and current regression coverage.

VERIFIED FINDING RECOMMENDATION / INFERRED NEXT STEP Verified findings are grounded in the current workspace code, docs, and tests. Recommendations are based on the remaining gaps that are visible in the current implementation.

## Executive Summary

The project is now materially capable of actionable exception classification. The reconciliation flow is deterministic from parse, normalize, rank, match, classify, summarize, export, and render. It emits accountant-facing categories with rule reasons, evidence, counterpart references, and recommended next actions through the API, workbook export, and web UI.

The most significant improvement since the prior review is that grouped residual cases are no longer taxonomy-only placeholders. The classifier now emits `LikelySplitCombineIssue` and `DepositGroupingIssue` from unmatched residual pools, and the test suite exercises those paths end-to-end. The largest remaining gap is architectural: grouped reasoning still happens only after a one-to-one greedy matcher leaves residuals, so support is bounded rather than complete.

> **Bottom line:** the platform has moved beyond generic unmatched reporting and now covers the full required taxonomy at runtime, but grouped categories remain partial because their logic is residual and size-limited rather than native to the matching engine.

## Current-State Findings

1. VERIFIED **High:** The system now classifies all required exception families somewhere in the runtime flow. In particular, the classifier contains residual grouped logic and residual timing logic before generic missing-item fallback, and those categories are validated in dedicated classifier and full-pipeline tests.
   Key modules: `packages/core/src/reporting/exceptionClassifier.ts`, `packages/core/src/reconcile.test.ts`, `packages/core/src/reporting/exceptionClassifier.test.ts`.
2. VERIFIED **Medium:** Summary and detail parity is materially stronger than in the prior review. The summary is built from final match results and final exceptions, and API tests now assert parity between exception counts, workbook Error Report category counts, and `summary.countsByCategory`.
   Key modules: `packages/core/src/reporting/summary.ts`, `apps/api/tests/reconciliation.test.ts`.
3. VERIFIED **Medium:** Reporting surfaces are accountant-usable. The workbook Error Report, API exceptions endpoint, and web exception table all expose category name, rule name, reason, evidence, next actions, and some counterpart context.
4. VERIFIED **High:** Grouped support is still bounded. The grouped classifier searches only one-to-many and many-to-one residual combinations up to three transactions on the grouped side, after the one-to-one matcher has already consumed candidates. This is strong enough for deterministic small grouped cases, but it is not a full grouped matching engine.

Key modules: `packages/core/src/matching/engine.ts`, `packages/core/src/reporting/exceptionClassifier.ts`, `docs/reconciliation-rules.md`.

5. VERIFIED **Medium:** There is still a maintainability warning in the classifier: the match-result classification block remains cognitively complex. This does not currently appear to be a functional bug, but it raises the cost of evolving exception precedence safely.
6. VERIFIED **Medium:** The codebase still contains a special `5703` calibration helper in summary code. The prior summary overwrite drift was addressed, but the legacy helper and provenance path remain and keep complexity high in reporting code.

## Classification Coverage Matrix

| Classification | Supported? | Evidence in Code | Gaps | Recommendation |
|---|---|---|---|---|
| timing difference | Yes | Extended-window matches can be promoted into TimingDifference, and unmatched residual same-amount timing-compatible pairs are classified before missing fallback. | Still residual rather than native to the matcher; transaction-kind rules remain selective. | Keep current residual stage and eventually move timing logic into a staged residual engine. |
| missing in bank | Yes | Unmatched QBO rows become MissingFromBank only after stronger categories are exhausted. | Fallback correctness still depends on earlier grouped and timing passes being complete enough. | Retain as a final fallback and preserve precedence discipline. |
| missing in QBO | Yes | Unmatched bank rows become MissingFromQBO only after stronger categories are exhausted. | Same limitation as MissingFromBank. | Retain as a final fallback and continue expanding stronger upstream categories. |
| duplicate in bank | Yes | Same-source duplicate detection runs before cross-source matching and is classified separately. | No deeper duplicate-versus-reversal distinction. | Keep current rule and add richer evidence only if operations require it. |
| duplicate in QBO | Yes | Same-source QBO duplicate detection is implemented and surfaced across summary and reporting. | No subreason taxonomy. | Keep deterministic duplicate rules as-is for now. |
| amount variance | Yes | The matcher emits AmountMismatch candidates | No explicit settlement-fee or | Add subreasons only if accounting review needs finer segmentation. |

| Classification | Supported? | Evidence in Code | Gaps | Recommendation |
|---|---|---|---|---|
| | | and the classifier/reporting surfaces preserve them. | partial-payment subtypes. | |
| date variance | Yes | Extended-window same-amount candidates remain DateVariance unless promoted to TimingDifference by later evidence. | Boundary with timing is still heuristic. | Keep current rule but make transaction-type evidence more explicit in the promotion path. |
| likely miscoding | Yes | Distinctive same-amount near-date contextual conflict becomes LikelyMiscoding with counterpart trace and next actions. | Still heuristic; not a true account-code diagnosis. | Preserve current rule and enrich with account or entity evidence if available later. |
| likely split/combine issue | Partial | The classifier now emits grouped split/combine exceptions from residual balanced one-to-many and many-to-one sets, and tests verify the path. | Limited to residual groups and to combinations of up to three grouped items. | Promote grouped logic into a first-class staged residual engine or native grouped matcher. |
| stale outstanding check | Yes | Unmatched aged QBO check-like disbursements are promoted to a dedicated stale category. | Check-like inference is still pattern-based. | Optionally allow account-specific stale thresholds and richer check typing. |
| deposit grouping issue | Partial | The classifier now emits grouped positive residual deposit-like sets as DepositGroupingIssue, with grouped evidence and tests. | Same size and staging limitations as split/combine logic; no native batching in the matcher. | Promote grouped deposit reasoning earlier in the residual decision pipeline. |

## Logic and Architecture Assessment

The current design supports clean deterministic classification better than it did in the previous review. Parsing, normalization, matching, classification, summary generation, export, and UI are separated cleanly enough that new categories were added without a broad rewrite.

The main architectural constraint remains the pairwise greedy matcher. It is still the primary decision engine, and grouped reasoning only happens after pairwise consumption has already removed candidates. That means the project now supports grouped categories in practical bounded cases, but not as a general matching capability. The next architectural step should be a staged residual engine that treats timing, grouped split/combine, grouped

deposits, miscoding, stale checks, and missing fallback as explicit phases rather than ad hoc tails on top of pairwise matching.

## Reporting and Auditability Assessment

The output is actionable for accounting staff. Exception rows expose the category, the originating rule, a human-readable reason, evidence, next actions, and source-row context. The workbook export and UI both preserve accountant-facing labels rather than developer shorthand.

Auditability is also meaningfully improved because summary and detail parity is now asserted across API and workbook output. The remaining auditability risk is completeness of grouped reasoning rather than lack of explanation. When the system chooses a category, it is usually explainable. The harder question is whether a richer grouped or timing category should have been chosen earlier.

## Prioritized Recommendations

### Immediate Improvements

**Title:** Refactor the match-result classification block in the classifier

**Problem observed:** The main match-result handling path in the classifier still carries a cognitive-complexity warning.

**Why it matters:** Exception precedence is now richer, and complex control flow increases regression risk when rules evolve.

**Severity:** Medium

**Implementation complexity:** Medium

**Recommended solution:** Extract promotion and recording subroutines for DateVariance, TimingDifference, LikelyMiscoding, and 5703-specific handling.

**Impacted files/modules:** `packages/core/src/reporting/exceptionClassifier.ts`

**Change type:** Logic, test coverage

**Suggested acceptance criteria:** Static analysis no longer flags the block for cognitive complexity and existing reconciliation classification tests still pass unchanged.

**Title:** Add near-miss grouped regression fixtures

**Problem observed:** Current grouped tests cover success cases but not enough negative boundaries.

**Why it matters:** Residual grouped logic can over-capture if amount, date, or type filters are loosened later.

**Severity:** Medium

**Implementation complexity:** Low

**Recommended solution:** Add fixtures where grouped totals almost balance, dates exceed the grouped window, or transaction kinds conflict and should remain missing-item exceptions.

**Impacted files/modules:** `packages/test-fixtures/src/sampleRows.ts`, `packages/core/src/reconcile.test.ts`, `packages/core/src/reporting/exceptionClassifier.test.ts`

**Change type:** Test coverage

**Suggested acceptance criteria:** Negative grouped fixtures do not emit grouped categories and fall through to the correct remaining bucket.

### Near-Term Enhancements

**Title:** Turn grouped residual logic into a staged residual engine

**Problem observed:** Grouped categories work, but only after one-to-one matching and only with a capped residual search.

**Why it matters:** Some valid grouped scenarios will still be consumed or fragmented before grouped analysis can see them.

**Severity:** High

**Implementation complexity:** High

**Recommended solution:** Preserve the current pairwise matcher for exact and review cases, then introduce an explicit staged residual analyzer that runs grouped searches before final missing fallback with configurable pool size and search bounds.

**Impacted files/modules:** `packages/core/src/matching/engine.ts`, `packages/core/src/reporting/exceptionClassifier.ts`, `packages/domain/src/types.ts`, `packages/config/src/default-config.ts`

**Change type:** Architectural, logic, data model, test coverage

**Suggested acceptance criteria:** Grouped cases can be traced to a dedicated staged decision family and support configurable search depth without breaking current pairwise regressions.

**Title:** Strengthen transaction-type evidence across all residual categories

**Problem observed:** Transaction-kind inference now exists, but its evidence is strongest in grouped and timing cases and weaker elsewhere.

**Why it matters:** Consistent transaction-type evidence improves deterministic separation between timing, miscoding, grouped receipts, stale checks, and missing items.

**Severity:** Medium

**Implementation complexity:** Medium

**Recommended solution:** Make transaction-kind compatibility or conflict a first-class field in rule traces for timing, grouped, stale-check, and miscoding classifications.

**Impacted files/modules:** `packages/core/src/reporting/exceptionClassifier.ts`, `packages/domain/src/types.ts`, `docs/reconciliation-rules.md`

**Change type:** Logic, data model, documentation, test coverage

**Suggested acceptance criteria:** Each residual exception family explicitly reports type evidence or type conflict in a stable rule-trace structure.

### Structural Redesign Items

**Title:** Retire legacy 5703-only complexity from reporting code

**Problem observed:** The reporting layer still carries a dedicated 5703 calibration helper and special provenance path.

**Why it matters:** Legacy special-case logic keeps complexity high and risks future summary or timing drift.

**Severity:** Medium

**Implementation complexity:** High

**Recommended solution:** Convert the remaining 5703-specific behavior into ordinary classifier rules or retire it once parity fixtures cover the same business cases.

**Impacted files/modules:** `packages/core/src/reporting/summary.ts`, `packages/core/src/reporting/exceptionClassifier.ts`, `packages/core/src/reporting/summary.edge-cases.test.ts`

**Change type:** Architectural, logic, documentation, test coverage

**Suggested acceptance criteria:** No special reporting override path remains that can diverge from final classified output, and legacy parity fixtures still pass.

### Quick Wins

- Add negative grouped fixtures so grouped logic is protected against false positives.
- Refactor the classifier's match-result block to reduce complexity without changing behavior.
- Keep API and workbook parity assertions in the regression suite as categories evolve.

### Medium-Term Refactors

- Elevate grouped and timing residual analysis into explicit staged processing.
- Expand transaction-type evidence across all residual categories.
- Reduce remaining 5703-only complexity by moving it into ordinary rule families.

### Strategic Redesign Items

- Move from pairwise-only matching plus residual heuristics to a staged residual decision engine.

- Make grouped reasoning a first-class capability rather than a residual classifier patch.
- Collapse legacy special-case reporting logic into general rule-driven reconciliation behavior.

## Suggested Rule Model

1. Exclude rows and classify invalid or missing data first.
2. Detect same-source duplicates before any cross-source matching.
3. Resolve deterministic one-to-one exact matches.
4. Resolve one-to-one review cases such as reference variance, description variance, amount variance, and ambiguity.
5. Run residual timing analysis on unmatched pools.
6. Run residual grouped split/combine analysis.
7. Run residual grouped deposit analysis.
8. Run likely miscoding analysis on distinctive same-amount near-date contextual conflicts.
9. Run stale outstanding check analysis for aged unmatched QBO check-like disbursements.
0. Fall back to MissingFromQBO and MissingFromBank only after stronger deterministic categories are exhausted.

The current implementation approximates this decision order already, but grouped support remains bounded because it is residual and size-limited. A future redesign should make this order explicit in a staged engine rather than distributed across pairwise matching and classifier tail logic.

## Testing Recommendations

- Add near-miss grouped fixtures where totals almost balance but should not emit grouped categories.
- Add fixtures where grouped candidates exceed the current three-item grouped-side limit so the bounded behavior is intentional and visible.
- Add targeted regressions for transaction-kind conflicts between deposit-like, transfer-like, and withdrawal-like residuals.
- Keep API integration tests asserting parity across exception endpoint output, workbook Error Report counts, and `summary.countsByCategory`.
- Add regression fixtures that prove 5703 summary behavior never drifts from final classified detail.
- Add UI tests verifying grouped evidence and next actions remain visible when grouped categories are present.

## Example Improved Output

| Source | File / Line | Category | Amount | Reason | Next Action |
| --- | --- | --- | --- | --- | --- |
| bank | bank.xlsx / 18 | Missing from QBO | -125.50 | Bank payment remained unmatched after exact, review, timing, grouped, miscoding, and stale-item analysis. | Verify whether the transaction was posted to the correct QBO account and period. |

| Source | File / Line | Category | Amount | Reason | Next Action |
|---|---|---|---|---|---|
| bank + qbo | bank.xlsx / 71 and qbo.xlsx / 74 | Timing Difference | -2,400.00 | Residual same-amount pair remained timing-compatible and posted outside the primary date tolerance but inside the timing window. | Carry as timing support and monitor the next reconciliation cycle. |
| bank + qbo | bank.xlsx / 85 and qbo.xlsx / 91 | Likely Miscoding | -500.00 | Same-amount near-date pair exists, but narrative or reference context conflicts materially. | Review whether the item was posted to the wrong vendor, customer, or account. |
| bank + qbo | bank.xlsx / 102 and qbo.xlsx / 103-104 | Likely Split/Combine Issue | -300.00 | One bank withdrawal aligns to two QBO withdrawals whose grouped total balances the bank line within tolerance. | Review the grouped counterpart set and document the one-to-many relationship. |
| bank + qbo | bank.xlsx / 118 and qbo.xlsx / 119-120 | Deposit Grouping Issue | 3,200.00 | One bank deposit aligns to multiple positive QBO receipts with deposit-like transaction kinds and a balanced grouped total. | Tie the bank deposit to the receipt batch and retain the deposit support package. |
| qbo | qbo.xlsx / 140 | Stale Outstanding Check | -150.00 | Unmatched QBO check-like disbursement is older than the stale threshold. | Determine whether to void, reissue, or keep the item outstanding. |

## Closing Assessment

The project now satisfies the core goal of turning reconciliation failures into meaningful accountant-facing categories instead of a generic unmatched dump. The current implementation is auditable, test-backed, and materially stronger than the prior review baseline. The next evolution is not to invent more categories, but to make grouped and timing reasoning first-class matching stages so bounded residual heuristics become a general capability.